

AI-103 SDK & Module Quick Reference — pip install import client key methods exam notes					
Service / SDK	pip install	Import & Client	Key Methods / Params	Code Pattern	Exam Notes
CORE — AUTHENTICATION					
Azure Identity (auth)	pip install azure-identity	from azure.identity import DefaultAzureCredential, ManagedIdentityCredential	Pass as credential= to any client DefaultAzureCredential() tries CLI → env vars → managed identity ManagedIdentityCredential(client_id=...) for user-assigned	from azure.core.credentials import AzureKeyCredential cred = AzureKeyCredential(os.getenv("KEY")) or cred = DefaultAzureCredential()	401 = bad auth · 403 = missing RBAC role (not a key issue) Production → managed identity. Dev → DefaultAzureCredential uses az login. User-assigned needs client_id=
DOMAIN 2 — GENERATIVE AI & AGENTS					
Azure OpenAI (chat / embed / images)	pip install openai	from openai import AzureOpenAI client = AzureOpenAI(azure_endpoint=ENDPOINT, api_key=KEY, api_version="2024-12-01-preview")	Chat: client.chat.completions.create(model, messages, temperature, max_tokens, response_format, seed, stream, tools) Embed: client.embeddings.create(model, input) → .data[0].embedding Image gen: client.images.generate(model, prompt, size, quality, n, output_format, background) Image edit: client.images.edit(model, image, mask, prompt, input_fidelity)	resp = client.chat.completions.create(model="my-gpt4o-deploy", messages=[{"role": "system", "content": "..."}, {"role": "user", "content": "..."}], temperature=0, max_tokens=500) print(resp.choices[0].message.content) print(resp.usage.total_tokens)	model= is DEPLOYMENT name not model name → 404 if wrong gpt-image returns b64_json NOT url Transparent bg → PNG only (no WebP on Azure) json_schema + strict=True = guaranteed schema json_object = valid JSON only Don't set temperature + top_p together
Foundry Agent Service (AIProjectClient)	pip install azure-ai-projects	from azure.ai.projects import AIProjectClient from azure.ai.projects.models import FileSearchTool, MessageRole client = AIProjectClient.from_connection_string(conn_str=CONN_STR, credential=DefaultAzureCredential())	create_agent(model, name, instructions, tools, tool_resources) create_thread() create_message(thread_id, role, content) create_and_process_run(thread_id, agent_id) ← waits list_messages(thread_id) upload_file_and_poll(file, purpose) create_vector_store_and_poll(file_ids, name)	agent = client.agents.create_agent(model="gpt-4o", ...) thread = client.agents.create_thread() client.agents.create_message(thread_id, role=MessageRole.USER, content="Q?") run = client.agents.create_and_process_run(thread_id=thread_id, agent_id=agent_id) msgs = client.agents.list_messages(thread_id)	Objects: Agent · Thread · Message · Run Run states: queued→in_progress→ completed requires_action failed requires_action = normal pause for YOUR custom fn Built-in tools: FileSearch, CodeInterpreter, BingGrounding, AzureAISearch Connection string from: Foundry → Project → Overview
Microsoft Agent Framework (MAF 1.0)	pip install agent-framework	from agent_framework import ChatAgent from agent_framework.azure import AzureAIAgentClient from azure.identity.aio import DefaultAzureCredential	ChatAgent(chat_client, instructions, tools={fn}) await agent.run(prompt) → .text Tools: plain Python fns with docstrings + type hints Workflows: sequential, concurrent, handoff, group, Magentic	async with DefaultAzureCredential() as cred: async with ChatAgent(chat_client=AzureAIAgentClient(project_endpoint=ENDPOINT, model_deployment_name="gpt-4o", async_credential=cred), instructions="...", tools=[my_fn]) as agent: r = await agent.run("...") print(r.text)	MAF is async-first — always async with + await Tool docstring = what the model reads to decide when to call it Vague docstring → tool never called SK/AutoGen = legacy predecessors (recognise terminology only) Workflow = deterministic graph (for auditable processes)
AI Evaluation	pip install azure-ai-evaluation	from azure.ai.evaluation import evaluate, GroundednessEvaluator, RelevanceEvaluator, CoherenceEvaluator	evaluate(data="file.jsonl", evaluators={...}, azure_ai_project=conn_str) Dataset fields: query, context, response, ground_truth AI-assisted evaluators need model_config	results = evaluate(data="eval.jsonl", evaluators={ "g": GroundednessEvaluator(model_cfg), "r": RelevanceEvaluator(model_cfg) }) score = results["metrics"]["g.gpt-groundedness"]	Groundedness + Relevance = AI-assisted (need LLM) F1/BLEU = mathematical (no LLM needed) Use in C/CD pipeline: sys.exit(1) if score drops
Content Safety	pip install azure-ai-contentssafety	from azure.ai.contentsafety import ContentSafetyClient from azure.ai.contentsafety.models import AnalyzeTextOptions, AnalyzeImageOptions, ImageData	client.analyze_text(AnalyzeTextOptions(text=...)) client.analyze_image(AnalyzeImageOptions(image=ImageData(content=bytes))) Categories: Hate Violence Sexual SelfHarm Severity: 0, 2, 4, 6	cs = ContentSafetyClient(endpoint, AzureKeyCredential(key)) r = cs.analyze_text(AnalyzeTextOptions(text="...")) for c in r.categories_analysis: if c.severity >= 4: # your threshold block()	Content Safety ≠ Content Filters Content Safety = standalone service you call explicitly Content Filters = built into every AOAI deployment (auto) Groundedness = not a CS feature → use Evaluation SDK Prompt Shieldness = indirect inject defence (separate)
DOMAIN 3 — VISION					
Azure AI Vision (Image Analysis)	pip install azure-ai-vision-imageanalysis	from azure.ai.vision.imageanalysis import ImageAnalysisClient from azure.ai.vision.imageanalysis.models import VisualFeatures	Features: CAPTION DENSE_CAPTIONS TAGS OBJECTS PEOPLE READ SMART_CROPS client.analyze_from_url(image_url, visual_features=[...]) client.analyze(image_data=bytes, visual_features=[...]) OCR: result.read.blocks()[].lines[].words[].confidence	vc = ImageAnalysisClient(endpoint, AzureKeyCredential(key)) r = vc.analyze_from_url("https://...", visual_features=[VisualFeatures.CAPTION, VisualFeatures.READ]) if r.caption: print(r.caption.text, r.caption.confidence) if r.read: for b in r.read.blocks: for l in b.lines: print(l.text)	Confidence lives on WORDS not lines Vision Read ≠ DI: multi-page PDF → DI · tables → DI Layout Image gen: gpt-image-2 (GA) or gpt-image-1 (Agent tool) gpt-image returns b64_json — decode with base64 Alt text → multimodal model with context prompt, not CAPTION
DOMAIN 4 — TEXT ANALYSIS & SPEECH					
Azure AI Language	pip install azure-ai-textanalytics	from azure.ai.textanalytics import TextAnalyticsClient client = TextAnalyticsClient(endpoint=ENDPOINT, credential=AzureKeyCredential(KEY))	recognize_entities(docs) → category, subcategory, confidence recognize_pii_entities(docs) → entities + doc.redacted_text analyze_sentiment(docs, show_opinion_mining=True) → positive/negative/neutral/mixed (doc only) extract_key_phrases(docs) detect_language(docs) → .primary_language.name	results = client.recognize_entities(["Satya Nadella is CEO of Microsoft."]) for doc in results: if doc.is_error: continue # ALWAYS CHECK for e in doc.entities: print(e.text, e.category, e.confidence_score) # PII + redaction for doc in client.recognize_pii_entities(["My SSN is 123-45-6789"]): print(doc.redacted_text)	Always check doc.is_error — batch ops succeed per-doc "mixed" sentiment = document level ONLY Opinion mining: show_opinion_mining=True → target + assessment Tone (urgency/hostility) → LLM, NOT Language service CLU replaced LUIS · QA = static KB · CLU = intent+entity
Azure AI Speech	pip install azure-cognitiveservices-speech	import azure.cognitiveservices.speech as speechsdk cfg = speechsdk.SpeechConfig(subscription=SPEECH_KEY, region=REGION)	STT: SpeechRecognizer(speech_config, audio_config) .recognize_once() → single utterance only .start_continuous_recognition() → long audio TTS: SpeechSynthesizer → speak_text_async(txt).get() or speak_ssm_async(ssml).get() SSML: <prosody rate="-10%"> <break time="400ms"/> <say-as interpret-as="digits"> <phoneme>	# Long audio — MUST use continuous rec.recognized.connect(lambda e: print(e.result.text)) rec.start_continuous_recognition() # SSML example synth.speak_ssml_async(""" <speak xmlns='...' xml:lang='en-IN'> <voice name='en-IN-NeerjaNeural'> <say-as interpret-as='digits'>4471</say-as> </voice></speak>""").get()	recognize_once() = SINGLE utterance + STOPS — exam bug pattern Long recording → start_continuous_recognition() + events Mispronounced brand → phoneme tag or custom lexicon 5 product names → phrase list (no training needed) Systematic domain → Custom Speech (training needed) Tone/frustration → Audio multimodal NOT STT+sentiment
DOMAIN 5 — INFORMATION EXTRACTION					
Document Intelligence	pip install azure-ai-formrecognizer	from azure.ai.formrecognizer import DocumentAnalysisClient client = DocumentAnalysisClient(endpoint=ENDPOINT, credential=AzureKeyCredential(KEY))	Prebuilt: prebuilt-invoice prebuilt-receipt prebuilt-idDocument prebuilt-layout prebuilt-read prebuilt-businessCard prebuilt-contract client.begin_analyze_document(model_id, document=f) → poller poller.result() → result.documents[].fields.get("FieldName") result.tables[].cells[]	with open("inv.pdf", "rb") as f: # "rb" binary! poller = client.begin_analyze_document("prebuilt-invoice", document=f) result = poller.result() # MUST call .result() for inv in result.documents: total = inv.fields.get("InvoiceTotal") if total: print(total.value, total.confidence)	begin * returns POLLER — must call .result() · open in "rb" mode Same layout → template · Varies → neural · Types→composed Tables + selection marks → Layout model Confidence on field: field.confidence → route low ones to human review Pro mode (objective) = now agentic mode in product
Azure AI Search	pip install azure-search-documents	from azure.search.documents import SearchClient from azure.search.documents.indexes import SearchIndexClient from azure.search.documents.models import VectorizedQuery, QueryType	Query: search_client.search(search_text, vector_queries=[VectorizedQuery(vector, k_nearest_neighbors, fields)], query_type=QueryType.SEMANTIC, semantic_configuration_name, select, top) Index: SearchField(..., searchable, filterable, facetable, sortable, retrievable, key) Vector field: vector_search_dimensions=1536 (must match embedding model)	results = sc.search(search_text=question, # keyword vector_queries=[VectorizedQuery(vector=embed(question), k_nearest_neighbors=5, fields="content_vector")], # vector query_type=QueryType.SEMANTIC, # rerank semantic_configuration_name="default", select=["title", "content", "url"], top=5) docs = list(results)	Keyword+Vector+Semantic = best RAG search Field attributes immutable after creation — must rebuild index Only STRING fields are searchable OCR skill = makes scanned PDFs searchable Split+Embedding skills = integrated vectorization Knowledge store ≠ index (→ Power BI, downstream analysis) Security filter = restrict results by user group
POLLER PATTERN (begin *) begin_analyze_document() → returns poller, NOT result poller.result() → blocks until done, returns result ✗ poller.documents → AttributeError (common exam bug) *_and_poll() → convenience: starts AND waits Same for: Sora 2 video = submit job → poll → download					
HTTP STATUS CODES 400 = Bad request (malformed input, file too large) 401 = Unauthenticated (bad key / expired token) 403 = Authenticated but no RBAC role assigned 404 = Wrong endpoint / deployment name 413 = Payload too large 429 = Rate limited → backoff; persistent → PTU					
ASYNC / AWAIT BUGS (MAF + Video)					

Prints <coroutine object>: missing await
async def + await fn() + asyncio.run(main())
azure.identity.aio for async credential (MAF)
Sora 2 video: submit → poll status → get gen_id → download
✖ Expecting video bytes from single call → fail